DS314: GENERATIVE AI & LLM SYSTEMS · ACADEMIC LAB MODULE

DS314 - Topic 5: Reasoning Prompts & Cognitive Scratchpads

Format: Jupyter Notebook (.ipynb) **Source:** Week 5 - Reasoning Prompts.ipynb **Render Engine:** KaTeX + MathML (Zero-Delim Math) **Academic Level:** 3rd Year BS Data Science

DS314 - Elective 1 (Generative AI)

Week 5 - Reasoning Prompts

Intended Learning Outcome (ILO): Explain why step-by-step reasoning can improve answers on multi-step problems, write reasoning prompts that end in an extractable final answer, use a worked example to control how the reasoning is formatted, compare direct and reasoning prompts with a scored test set (accuracy, tokens, and time), and describe how built-in "thinking" in modern models changes when reasoning prompts are needed.

Overview: In Week 4, you learned to write clear prompts and test them with a labeled set. Those tasks (classifying a message, writing an ad) mainly required the model to understand and generate an answer. This week, we look at a different kind of task: **problems that require several steps to solve**, such as computing a bill, checking whether a customer qualifies for a refund, or planning a schedule. These are tasks where a model may produce a confident answer even when one of the steps is incorrect.

What you'll build

- A helper that reports the model's reply, token usage, and response time
- LangChain `ChatPromptTemplate` prompts for direct and step-by-step responses
- A function that pulls the final answer out of a longer response so code can use it
- A scored comparison of direct and reasoning prompts
- An experiment that observes whether the API reports any built-in "thinking" for direct prompts
- A worked-example prompt that controls the format of the reasoning

Prerequisites

- The virtual environment and `.env` file from Week 2, with `langchain-google-genai` and `python-dotenv` installed
- Your venv selected as the notebook kernel
- Your `.env` file containing `GEMINI_API_KEY=...`, the key name used throughout this course
- The `ChatGoogleGenerativeAI`, `ChatPromptTemplate`, `StrOutputParser`, `invoke()`, `ask()`, and `evaluate()` ideas from Week 4

Setup: The Helper, Now With Usage Tracking

We reuse the `ask()` helper from Week 4 and add a second helper, `ask_with_usage()`. Both use the same LangChain-based model setup (`ChatGoogleGenerativeAI`) you learned in Week 4.

The difference is that `ask()` returns only the model's reply, while `ask_with_usage()` returns the reply **and** a small usage report. The report includes input tokens, output tokens, thinking tokens when the API reports them, and response time.

LangChain attaches the token counts to the reply as `usage_metadata`. Thinking tokens are reported inside it as `output_token_details["reasoning"]`, and they are already **counted within** `output_tokens`. Our helper copies that value into a simple `thinking_tokens` field, or records 0 if the model did not report one.

We also keep the prompt layer in LangChain. This week, `ChatPromptTemplate` objects are passed into the helper together with the values that fill their `{variables}`.

Where you can use this: Production LLM applications can log token usage and response time for requests. This helps teams identify which features use more resources, detect prompts that become slower after an edit, and estimate usage costs.

In [2]:

```
import os
import re
import time

from dotenv import load_dotenv
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.messages import HumanMessage, SystemMessage
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv()

# Get the Gemini API key from the .env file
api_key = os.getenv("GEMINI_API_KEY")

if not api_key:
    raise ValueError(
        "GEMINI_API_KEY not found. Check that your .env file is named exactly '.env' "
        "and sits in the same folder as this notebook."
    )

# Choose the Gemini model used throughout this notebook
MODEL_ID = "gemini-3.1-flash-lite"

# Create the default LangChain model object
llm = ChatGoogleGenerativeAI(
    model=MODEL_ID,
    google_api_key=api_key,
)

# Converts a LangChain AIMessage into plain text
parser = StrOutputParser()


def _generate(
    prompt,
    variables=None,
    system=None,
    temperature=None,
    max_tokens=None,
    model=None,
    thinking_level=None,
    timing=None,
):
    """Send one request through LangChain and return the raw AIMessage."""

    # Use the default model unless custom settings are provided
    current_llm = llm

    if (
        model
        or temperature is not None
        or max_tokens is not None
        or thinking_level is not None
    ):
        current_llm = ChatGoogleGenerativeAI(
            model=model or MODEL_ID,
            google_api_key=api_key,
            temperature=temperature,
            max_tokens=max_tokens,
            thinking_level=thinking_level,
        )

    # Turn the prompt into a list of LangChain messages
    if isinstance(prompt, ChatPromptTemplate):
```

```
        messages = prompt.invoke(
            variables or {}
        ).to_messages()
    else:
        messages = [HumanMessage(content=prompt)]

    # Add a system instruction when one is provided
    if system:
        messages = [SystemMessage(content=system)] + messages

    # Try up to three times in case of a temporary rate-limit error
    for attempt in range(3):
        try:
            # Time only the request itself, so a wait before a retry is not counted
            started = time.perf_counter()
            response = current_llm.invoke(messages)

            if timing is not None:
                timing["seconds"] = time.perf_counter() - started

            return response

        except Exception as e:
            if "429" in str(e) and attempt < 2:
                time.sleep(20)
                continue

            raise


def _text_of(response):
    """Extract plain text from the LangChain AIMessage."""

    return parser.invoke(response)


def ask(prompt, variables=None, **settings):
    """Send one prompt and return only the model's text reply."""

    return _text_of(
        _generate(
            prompt,
            variables=variables,
            **settings
        )
    )


def ask_with_usage(prompt, variables=None, **settings):
    """Send one prompt and return the reply plus usage information."""

    # _generate() records the response time of the successful request here
    timing = {}

    # Keep the complete AIMessage because it contains usage metadata
    response = _generate(
        prompt,
        variables=variables,
        timing=timing,
        **settings
    )

    # Response time of the request
    seconds = timing.get("seconds", 0.0)

    # LangChain stores token information in usage_metadata
```

```
    usage_metadata = response.usage_metadata or {}

    # Read reported thinking/reasoning tokens when available
    output_details = (
        usage_metadata.get("output_token_details") or {}
    )

    thinking_tokens = (
        output_details.get("reasoning", 0) or 0
    )

    usage = {
        "prompt_tokens": usage_metadata.get(
            "input_tokens", 0
        ),
        "output_tokens": usage_metadata.get(
            "output_tokens", 0
        ),
        "thinking_tokens": thinking_tokens,
        "seconds": seconds,
    }

    return _text_of(response), usage


# Test the helper with a simple string prompt
reply, usage = ask_with_usage(
    "What is 15% of 200? Reply with only the number."
)

print("Reply:", reply)
print("Usage:", usage)
```

Out [2]:

```
Reply: 30
Usage: {'prompt_tokens': 19, 'output_tokens': 2, 'thinking_tokens': 0, 'seconds': 1.0421883999952115}
```

Function	Purpose
`_generate()`	Talk to the LLM.
`_text_of()`	Give me just the text.
`ask()`	Talk to the LLM and give me text.
`ask_with_usage()`	Talk to the LLM and give me text + measurements.

What just happened:

- `_generate()` sends the request through LangChain's `ChatGoogleGenerativeAI` model using `invoke()`. It returns the model's complete `AIMessage`, which contains the response text and usage metadata when the model provides it.
- `ask()` uses `_generate()` and `StrOutputParser()` to return only the model's text. `ask_with_usage()` keeps the complete `AIMessage` long enough to read its `usage_metadata`, then returns the text together with a small usage report.
- The helper accepts either a plain string or a LangChain `ChatPromptTemplate`. A plain string becomes a single `HumanMessage`. A `ChatPromptTemplate` fills in its `{variables}` when `_generate()` calls its `invoke()` method, and the resulting messages are sent to the model.
- The usage report lets us measure token usage and response time. Response time covers the model request itself. If the model reports thinking tokens (`output_token_details["reasoning"]`), we capture those too. Because they are part of `output_tokens`, `thinking_tokens` can never be larger than `output_tokens`.

Checkpoint: The reply was just a number. Look at the usage report. How many output tokens did the model use? How long did the request take? If `thinking_tokens` is greater than 0, how many were reported?

Answer: Because the reply is only a number, the output should be very small, just a few tokens. The request typically takes a few seconds, and that time varies from one request to the next. `thinking_tokens` is usually 0 here, which means no thinking-token usage was reported for this response. Record your own values, because later we will compare these measurements between a direct prompt and a reasoning prompt.

1. Why Writing the Steps Out Can Help

Recall Week 1: an LLM generates its response **one token at a time**, with each generated token becoming part of the context for the tokens that come after it. This matters when a problem requires several steps.

- If you ask for an immediate answer ("Reply with only the number"), the model has to produce the final answer without writing out the intermediate steps.
- If you ask the model to work through the problem step by step, the intermediate results (the fare for one student, the discounted price, etc.) become part of the generated response. Later parts of the response can use those results when continuing the solution.

A real-life comparison: your teacher says "show your solution." A student who does the whole calculation in their head might lose track of a step. A student who writes each line on paper can see each intermediate result and use it in the next calculation. A model's generated text can serve a similar role when it is asked to work through a problem step by step.

Where you can use this: Any task where the answer depends on several calculations or rules: computing a phone bill with overage charges, checking whether a customer qualifies for a refund (like the Week 3 refund policy, which combined "within 30 days" with "custom-made items"), scheduling around several constraints, or checking a discount stacked on another discount.

Checkpoint: Why can asking a model to write out its steps help with a multi-step problem?

Answer: Writing out intermediate results gives the model's later generation access to those results as part of the context. Instead of producing only the final answer immediately, the model can build the solution step by step.

2. Direct vs. Step-by-Step Prompts

Checkpoint: what are we adding, and why?

We take the prompt anatomy from Week 4 and add one instruction to the **format** component: *show the steps first, then give the answer.* Nothing else changes. This lets us compare the two prompting approaches fairly.

This time, we use LangChain's `ChatPromptTemplate` so the prompts follow the same prompt-template approach you learned in Week 4.

Our problem set has six short word problems. Each one has an answer key that we compute **in code** rather than typing the number by hand. That way, the key can be checked by the program, and you'll use the same approach in the lab.

The important idea is that **the problem stays the same**. We are changing only how we ask the model to produce its answer.

Where you can use this: Bill and fee calculators inside a telco or bank assistant, "how much will I pay?" questions in a store or delivery app, grading assistants that check a student's working, and any FAQ bot where the answer requires arithmetic.

In [3]:

```
problems = [
    {
        "problem": "A jeepney fare is ₱13 for the first 4 km, plus ₱2 for every additional km. "
                   "Three students ride 10 km together, and each gets a 20% discount on their own fare. "
                   "How much do the three students pay in total, in pesos?",
        # Python calculates the expected answer for us.
        "answer": (13 + 2 * (10 - 4)) * 0.8 * 3,
    },
    {
        "problem": "Aling Nena's store sells chips at ₱12 per pack, or 3 packs for ₱30. "
                   "Aling Nena buys exactly 11 packs and uses the 3-for-₱30 deal as many times as possi
                   "How much does she pay in total, in pesos?",
        "answer": 3 * 30 + 2 * 12,
    },
    {
        "problem": "A postpaid plan costs ₱299 per month and includes 15 GB of data. "
                   "Each extra whole GB costs ₱50. Maria used 22 GB this month. "
                   "What is her total bill, in pesos?",
        "answer": 299 + (22 - 15) * 50,
    },
    {
        "problem": "A Jollibee branch sold 240 Chickenjoy meals on Monday. On Tuesday it sold 25% more
                   "On Wednesday it sold 20% fewer than on Tuesday. How many Chickenjoy meals did it se
        "answer": 240 * 1.25 * 0.8,
    },
    {
        "problem": "A student buys ₱100 of mobile load. A 15% service fee is taken out first. "
                   "The remaining amount buys data at ₱5 per 100 MB. How many MB of data does the stude
        "answer": (100 * 0.85) / 5 * 100,
    },
    {
        "problem": "In a class of 40 students, 60% passed the quiz. Of those who passed, one quarter go
                   "How many students in the whole class did NOT get a perfect score?",
        "answer": 40 - (40 * 0.6) * 0.25,
    },
]



# Direct prompt:
# Ask the model to jump straight to the final answer.
direct_prompt = ChatPromptTemplate.from_template(
    """Solve the problem below. Reply with ONLY the final number, nothing else.

Problem: {problem}"""
)


# Reasoning prompt:
# Ask the model to show its calculations before giving the final answer.
reasoning_prompt = ChatPromptTemplate.from_template(
    """Solve the problem below step by step. Show each calculation on its own line.
On the last line, write exactly: ANSWER: <number>

Problem: {problem}"""
)


# Use one problem to demonstrate the difference between the two prompts.
demo = problems[0]["problem"]

print("EXPECTED ANSWER:", problems[0]["answer"])

print()
print("--- DIRECT PROMPT ---")
```

```
print(ask(direct_prompt, {"problem": demo}))

print()
print("--- REASONING PROMPT ---")
print(ask(reasoning_prompt, {"problem": demo}))
```

Out [3]:

```
EXPECTED ANSWER: 60.0

--- DIRECT PROMPT ---
60

--- REASONING PROMPT ---
Additional distance beyond the first 4 km = 10 km - 4 km = 6 km
Fare for the additional distance = 6 km * ₱2/km = ₱12
Total fare for one student = ₱13 + ₱12 = ₱25
Discount per student = 20% of ₱25 = 0.20 * ₱25 = ₱5
Discounted fare for one student = ₱25 - ₱5 = ₱20
Total fare for three students = 3 * ₱20 = ₱60
ANSWER: 60
```

What just happened: Both `ChatPromptTemplate` objects contain the same `{problem}` variable. The only difference is the instruction: "reply with only the number" vs. "show each calculation, then write `ANSWER:`." The second reply is longer because it contains the working.

Notice the details in this problem: the ₱2 charge applies only to the **6 km beyond the first 4 km**, not to all 10 km, and the 20% discount applies to **each student's own fare** before multiplying by three. Reading the steps lets you see *how* the model reached its number, and whether it applied each rule to the right amount.

However, showing the steps does not automatically mean the model's reasoning is correct. A model can produce a convincing-looking sequence of calculations and still make a mistake. That is why we will test both approaches on multiple problems.

3. Making the Final Answer Easy for Code to Find

Checkpoint: what are we adding, and why?

A long step-by-step reply is useful for a person, but a program needs **one value** it can extract and use. If we ask the model to always end with `ANSWER: <number>`, our code can find that line reliably.

This is the same idea as Week 4's "reply with the category name only," applied to reasoning: **let the model show its solution in a readable format, then require it to commit to the final answer in a predictable format.**

This is still plain text, not a formal structured-output format. We are simply giving the program a predictable pattern to look for. Week 6 will take this idea further with proper structured output such as JSON.

Where you can use this: Any app where a value needs to go into other code: a fee calculator that fills in a form, an eligibility checker that returns yes or no to a workflow, or a grading assistant that produces a score to store in a spreadsheet.

In [4]:

```
def extract_number(text):
    """Pull the final numeric answer out of a model reply.

    First looks for a line like 'ANSWER: 1,700'.
    If the model forgot that line, falls back to the last number in the reply.
    """

    # Look specifically for the ANSWER: label followed by a number.
    match = re.search(
        r"ANSWER:\s*[₱$]?\s*(-?[\d,]*\.?\d+)",
        text,
        flags=re.IGNORECASE
    )

    if match:
        # Get only the number captured by the regular expression.
        raw = match.group(1)

    else:
        # If there is no ANSWER: line, find all numbers in the reply.
        numbers = re.findall(
            r"-?[\d,]*\.?\d+",
            text
        )

        # No number means there is nothing to extract.
        if not numbers:
            return None

        # Use the last number found in the reply.
        raw = numbers[-1]

    try:
        # Remove commas and convert the result into a Python float.
        return float(raw.replace(",", ""))

    except ValueError:
        # Return None if the extracted text cannot be converted to a number.
        return None


# Try the function on a few different model replies
samples = [
    "Step 1: 100 x 0.85 = 85\n"
    "Step 2: 85 / 5 = 17\n"
    "Step 3: 17 x 100 = 1,700\n"
    "ANSWER: 1,700",

    "1700",

    "The student gets 1700 MB.",

    "I am not sure.",
]


for sample in samples:

    result = extract_number(sample)

    print("MODEL REPLY:")
    print(sample)

    print("EXTRACTED NUMBER:", result)
```

```
    print("-" * 50)
```

Out [4]:

```
MODEL REPLY:
Step 1: 100 x 0.85 = 85
Step 2: 85 / 5 = 17
Step 3: 17 x 100 = 1,700
ANSWER: 1,700
EXTRACTED NUMBER: 1700.0
--------------------------------------------------
MODEL REPLY:
1700
EXTRACTED NUMBER: 1700.0
--------------------------------------------------
MODEL REPLY:
The student gets 1700 MB.
EXTRACTED NUMBER: 1700.0
--------------------------------------------------
MODEL REPLY:
I am not sure.
EXTRACTED NUMBER: None
--------------------------------------------------
```

What just happened: `re.search` looks for the text `ANSWER:` followed by a number and returns that number. Commas (`1,700`) and a peso sign are handled. If the model skipped the `ANSWER:` line, the function falls back to the **last number** in the reply. This fallback is less reliable because the last number is not necessarily the final answer. If there is no number at all, it returns `None`, meaning that our code could not extract an answer.

Checkpoint: Why do we ask for `ANSWER: <number>` on the last line, instead of just reading whatever the model wrote?

Answer: Free text varies from run to run ("The total is...", "So she pays...", numbers buried in sentences). A fixed final line gives the code one predictable place to look, so a parsing problem is less likely to be mistaken for a wrong answer.

4. Scoring the Two Prompts

Checkpoint: what are we adding, and why?

"It seemed to work" isn't evidence. Following the Week 4 habit of testing on more than one input, we run **both** templates over a small set of problems and record several things: **accuracy**, **average output tokens**, **average thinking tokens when reported**, and **average seconds**.

Accuracy tells you whether the prompt produced correct answers. Tokens and seconds show how much output and processing time each approach used. Output tokens include any thinking tokens the model reports, and the separate thinking-token count, when available, shows how much of that output was internal processing.

We use a set of ten harder multi-step problems (four medium and six hard). Each prompt is run on all ten, so this experiment sends 20 requests in total. The same evaluation can be run on a larger dataset later.

Where you can use this

This kind of comparison helps when deciding, with numbers, whether a feature should use a long reasoning prompt or a short one. A team that answers 100,000 questions a month will care about differences in response length, reported thinking effort, and processing time.

In [5]:

```
challenge_problems = [
    {
        "problem": (
            "A store gives a 15% discount on an appliance priced at ₱8,000. "
            "After the discount, a 12% service fee is added to the discounted price. "
            "The customer pays ₱5,000 upfront and the rest in equal monthly payments "
            "over 5 months. How much is each monthly payment, in pesos?"
        ),
        "answer": ((8000 * 0.85) * 1.12 - 5000) / 5,
        "difficulty": "medium",
    },

    {
        "problem": (
            "A delivery company charges ₱80 for the first 5 km and ₱15 for every "
            "additional km. A customer orders 3 packages delivered to the same address. "
            "The first package is charged the full delivery fee, while the second and "
            "third packages each receive a 20% discount on the delivery fee. "
            "The total distance is 13 km. How much is the total delivery charge?"
        ),
        "answer": (
            (80 + 15 * (13 - 5))
            + (80 + 15 * (13 - 5)) * 0.8
            + (80 + 15 * (13 - 5)) * 0.8
        ),
        "difficulty": "medium",
    },

    {
        "problem": (
            "A school has 480 students. On Monday, 75% attended class. "
            "On Tuesday, attendance was 10% lower than Monday's attendance. "
            "On Wednesday, attendance increased by 25% compared with Tuesday. "
            "How many students attended on Wednesday?"
        ),
        "answer": 480 * 0.75 * 0.90 * 1.25,
        "difficulty": "medium",
    },

    {
        "problem": (
            "A restaurant normally sells a meal for ₱250. During a promotion, "
            "the restaurant first gives a 20% discount. A 10% service charge is then "
            "added to the discounted price. A customer has a ₱50 voucher that is "
            "applied after the service charge. How much does the customer pay?"
        ),
        "answer": (250 * 0.80) * 1.10 - 50,
        "difficulty": "medium",
    },

    {
        "problem": (
            "A company has 600 employees. 40% work remotely. Of the employees who "
            "work remotely, 30% work remotely only three days per week. Of the remaining "
            "remote employees, 25% work remotely only two days per week. "
            "How many employees work remotely more than three days per week?"
        ),
        "answer": (600 * 0.40) * (1 - 0.30) * (1 - 0.25),
        "difficulty": "hard",
    },

    {
        "problem": (
            "A phone originally costs ₱18,000. The store increases the price by 15%, "
            "then offers a 20% discount on the increased price. A customer also receives "
```

```
            "a ₱1,000 voucher. Compared with the original ₱18,000 price, how much does "
            "the customer actually save?"
        ),
        "answer": 18000 - (18000 * 1.15 * 0.80 - 1000),
        "difficulty": "hard",
    },

    {
        "problem": (
            "A company receives 1,200 applications. 35% are rejected during the first "
            "screening. Of the remaining applications, 40% pass the technical test. "
            "Of those who pass the technical test, 25% are rejected during the final "
            "interview. How many applicants are finally accepted?"
        ),
        "answer": 1200 * 0.65 * 0.40 * 0.75,
        "difficulty": "hard",
    },

    {
        "problem": (
            "A water tank is 60% full and contains 720 liters. During the morning, "
            "15% of the water currently in the tank is used. In the afternoon, "
            "240 liters are added. The tank has a capacity of 1,200 liters. "
            "What percentage of the tank is full at the end of the day?"
        ),
        "answer": ((720 * 0.85) + 240) / 1200 * 100,
        "difficulty": "hard",
    },

    {
        "problem": (
            "A store has 240 notebooks. On Monday, it sells 30% of its notebooks. "
            "On Tuesday, it sells 25% of the notebooks that remained after Monday. "
            "On Wednesday, it receives 54 new notebooks and then sells 40% of its "
            "new total inventory. How many notebooks remain after Wednesday?"
        ),
        "answer": ((240 - 240 * 0.30) * 0.75 + 54) * 0.60,
        "difficulty": "hard",
    },

    {
        "problem": (
            "A training program starts with 500 applicants. After the first screening, "
            "20% are removed. Of those remaining, 30% fail the first assessment. "
            "Of those who pass, 10% withdraw before the final assessment. "
            "Finally, 75% of those who take the final assessment pass it. "
            "How many applicants complete the program successfully?"
        ),
        "answer": 500 * 0.80 * 0.70 * 0.90 * 0.75,
        "difficulty": "hard",
    },
]
```

In [6]:

```python
import time


def evaluate_numeric(prompt, dataset, delay=1):
    """Run one ChatPromptTemplate over a set of numeric problems.

    Returns:
        accuracy: proportion of correct answers
        rows: detailed results for every problem
    """

    rows = []

    # Test the same problems one by one.
    for i, item in enumerate(dataset, start=1):

        print(f"Processing {i}/{len(dataset)}...", end="\r")

        # Pass the prompt and its variable to the helper.
        # The helper fills in {problem} and sends the prompt to the LLM.
        reply, usage = ask_with_usage(
            prompt,
            {"problem": item["problem"]}
        )

        # Convert the model's text reply into a number.
        predicted = extract_number(reply)

        # Compare the extracted number with our known answer.
        correct = (
            predicted is not None
            and abs(predicted - item["answer"]) < 0.01
        )

        # Store the result and usage information for later analysis.
        rows.append({
            "expected": item["answer"],
            "predicted": predicted,
            "correct": correct,
            "difficulty": item.get("difficulty", ""),
            **usage,
        })

        # Pause briefly before the next API request.
        if i < len(dataset):
            time.sleep(delay)

    print()

    # Calculate the proportion of correct answers.
    accuracy = sum(r["correct"] for r in rows) / len(rows)

    return accuracy, rows


def summarize(name, accuracy, rows):
    """Print the overall results for one prompt."""

    n = len(rows)

    # Calculate the average metrics across all problems.
    avg_output = sum(
        r["output_tokens"] for r in rows
    ) / n
```

```
    avg_thinking = sum(
        r["thinking_tokens"] for r in rows
    ) / n

    avg_seconds = sum(
        r["seconds"] for r in rows
    ) / n

    print(f"{name}")
    print(f"  Accuracy:            {accuracy:.0%}")
    print(f"  Avg output tokens:   {avg_output:.0f} (includes thinking tokens)")
    print(f"  Avg thinking tokens: {avg_thinking:.0f}")
    print(f"  Avg response time:   {avg_seconds:.1f} seconds")

    print()
    print("Individual results:")

    # Show what happened on every problem.
    for i, r in enumerate(rows, start=1):

        mark = "OK" if r["correct"] else "WRONG"

        print(
            f"  {i}. [{mark}] "
            f"Expected: {r['expected']:g} | "
            f"Got: {r['predicted']}"
        )


# ----------------------------------------------------
# CLASSROOM EXPERIMENT
# ----------------------------------------------------

# Use the new challenge set for the actual comparison.
test_problems = challenge_problems


# Test the DIRECT prompt.
direct_acc, direct_rows = evaluate_numeric(
    direct_prompt,
    test_problems
)

print()


# Test the STEP-BY-STEP prompt on the exact same problems.
reason_acc, reason_rows = evaluate_numeric(
    reasoning_prompt,
    test_problems
)

print()


# Display the results for both approaches.
print("=" * 55)
print("DIRECT PROMPT")
print("=" * 55)

summarize(
    "DIRECT",
    direct_acc,
    direct_rows
)
```

```
print()
print("=" * 55)
print("STEP-BY-STEP PROMPT")
print("=" * 55)

summarize(
    "STEP-BY-STEP",
    reason_acc,
    reason_rows
)
```

Out [6]:

```
Processing 10/10...

Processing 10/10...

=======================================================
DIRECT PROMPT
=======================================================
DIRECT
  Accuracy:            40%
  Avg output tokens:   3 (includes thinking tokens)
  Avg thinking tokens: 0
  Avg response time:   3.1 seconds

Individual results:
  1. [WRONG] Expected: 523.2 | Got: 1152.0
  2. [WRONG] Expected: 520 | Got: 504.0
  3. [OK] Expected: 405 | Got: 405.0
  4. [OK] Expected: 170 | Got: 170.0
  5. [WRONG] Expected: 126 | Got: 180.0
  6. [WRONG] Expected: 2440 | Got: 1360.0
  7. [WRONG] Expected: 234 | Got: 351.0
  8. [OK] Expected: 71 | Got: 71.0
  9. [WRONG] Expected: 108 | Got: 153.0
  10. [OK] Expected: 189 | Got: 189.0

=======================================================
STEP-BY-STEP PROMPT
=======================================================
STEP-BY-STEP
  Accuracy:            100%
  Avg output tokens:   160 (includes thinking tokens)
  Avg thinking tokens: 0
  Avg response time:   3.9 seconds

Individual results:
  1. [OK] Expected: 523.2 | Got: 523.2
  2. [OK] Expected: 520 | Got: 520.0
  3. [OK] Expected: 405 | Got: 405.0
  4. [OK] Expected: 170 | Got: 170.0
  5. [OK] Expected: 126 | Got: 126.0
  6. [OK] Expected: 2440 | Got: 2440.0
  7. [OK] Expected: 234 | Got: 234.0
  8. [OK] Expected: 71 | Got: 71.0
  9. [OK] Expected: 108 | Got: 108.0
  10. [OK] Expected: 189 | Got: 189.0
```

What should we notice?

- Compare **accuracy** first. Did step-by-step get more problems right?
- Then compare **output tokens** and **response time**. The step-by-step reply should be much longer because it was instructed to show its calculations. However, response time can vary between requests, so a longer response does not necessarily mean a longer wait.
- Look at the **thinking tokens** line. If thinking tokens are reported for the direct prompt, the model performed reported internal processing even though we asked for only a number. Remember that thinking tokens are already included in the output-token average. We will explore this in Section 5.

What should we conclude?

Base your conclusions on the numbers from **your own run**, not on a general rule. Here is how to read the results:

- If **both prompts have the same accuracy**, these problems may be easy enough for this model that visible steps made no difference in correctness.
- If **step-by-step scored higher**, the written working helped on at least some of these problems. Check which ones, and look at the reply to see why.
- Expect the step-by-step prompt to use **more output tokens**, since it writes out its calculations. Response time may go either way. It varies between requests, so a single run is weak evidence, and a larger, repeated test would be stronger.
- If `thinking_tokens` is **0** for both approaches, no thinking-token usage was reported in these responses. This does **not** prove that the model did no internal processing.

Checkpoint: Suppose both templates scored 100%. Does that prove reasoning prompts are useless?

Answer: No. It only shows that *these ten problems* were easy enough for this model to solve correctly with either prompt. To learn more, you would test harder problems with more steps, distractors, or conditions. You would also compare token usage and response time, because the reasoning prompt costs extra output.

The important lesson is: **measure the behavior you care about instead of assuming what a prompt will do.**

5. Built-In "Thinking": Reasoning You Don't Write

Checkpoint: What are we adding?

So far, our reasoning prompts have asked the model to **show its reasoning in the response**.

For example:

> *Solve the problem step by step and show your calculations.*

Some models can also perform **additional internal processing** before producing their final answer. This is what we refer to as **built-in thinking**.

	Prompted reasoning	**Built-in thinking**
What is it?	Reasoning we explicitly ask the model to show	Additional processing performed by the model
Do we see it?	Yes, in the response	Usually not
Can usage be reported?	As normal output tokens	As thinking/reasoning tokens, when reported

The important distinction is:

> ***Prompted reasoning = reasoning we ask the model to show.***
> ***Built-in thinking = internal processing the model may perform before answering.***

These can happen at the same time. A model may internally process a problem and also provide a step-by-step explanation because we asked for one.

Important: If our `thinking_tokens` value is `0`, this means **no thinking-token usage was reported by our current setup**. It does not prove that the model performed no internal processing.

Where does this matter?

When developing an LLM application, we may want to know whether a task actually benefits from additional reasoning. A simple task may be handled directly, while a more difficult task may benefit from additional processing.

Direct Prompt:

In [7]:
```
hard = challenge_problems[5]["problem"]

reply, usage = ask_with_usage(
    direct_prompt,
    {"problem": hard}
)

print("Reply:", reply)
print("Expected:", challenge_problems[5]["answer"])

print()

print(
    f"Output tokens: {usage['output_tokens']} | "
    f"Thinking tokens: {usage['thinking_tokens']} | "
    f"Seconds: {usage['seconds']:.1f}"
)
```

Out [7]:
```
Reply: 1360
Expected: 2440.0

Output tokens: 4 | Thinking tokens: 0 | Seconds: 1.3
```

Reasoning Prompt:

In [8]:

```
hard = challenge_problems[5]["problem"]

reply, usage = ask_with_usage(
    reasoning_prompt,
    {"problem": hard}
)

print("Reply:", reply)
print("Expected:", challenge_problems[5]["answer"])

print()

print(
    f"Output tokens: {usage['output_tokens']} | "
    f"Thinking tokens: {usage['thinking_tokens']} | "
    f"Seconds: {usage['seconds']:.1f}"
)
```

Out [8]:

```
Reply: 15% increase of ₱18,000 = 0.15 * 18,000 = ₱2,700
New price after increase = 18,000 + 2,700 = ₱20,700
20% discount on the new price = 0.20 * 20,700 = ₱4,140
Price after discount = 20,700 - 4,140 = ₱16,560
Price after applying the ₱1,000 voucher = 16,560 - 1,000 = ₱15,560
Savings compared to the original price = 18,000 - 15,560 = ₱2,440

ANSWER: 2440
Expected: 2440.0

Output tokens: 199 | Thinking tokens: 0 | Seconds: 1.7
```

What should we notice?

Look at the `thinking_tokens` value.

If the API reports a value greater than 0, the model reported using tokens for internal thinking even though we asked it to reply with only the answer.

If it reports 0, that means **no thinking-token usage was reported for this response**. It does not prove that the model performed no internal processing.

Also compare `Reply` with `Expected`. This problem is labeled hard in the set, so it is a good test of whether a direct answer is enough.

This is why we distinguish between **what the model visibly writes** and **what the API reports about the model's processing**.

Checkpoint: If the direct prompt already produces reported thinking tokens, what does that tell us about the usefulness of adding "think step by step" to every prompt?

Answer: It tells us that asking for visible step-by-step reasoning is not the only way a model can perform additional processing. Whether the extra prompt instruction helps still depends on the model and the task.

In [9]:

```
reasoning_test = """
There are three boxes labeled A, B, and C.

One box contains only apples.
One box contains only oranges.
One box contains both apples and oranges.

All three labels are incorrect.

You may take exactly one fruit from exactly one box without looking inside.

Which box should you take a fruit from, and how can you determine the correct labels of all three boxes
"""

reply, usage = ask_with_usage(
    reasoning_test,
    thinking_level="high"
)

print("Reply:")
print(reply)

print()

print(
    f"Output tokens: {usage['output_tokens']} | "
    f"Thinking tokens: {usage['thinking_tokens']} | "
    f"Seconds: {usage['seconds']:.1f}"
)
```

Out [9]:

```
Reply:
You should pick a fruit from the box labeled **"Apples and Oranges" (Mixed)**.

Here is the step-by-step logic to determine the contents of all three boxes:

### The Key Insight
You know that **all** labels are incorrect. This means the box labeled "Mixed" **cannot** actually
contain a mix of apples and oranges. It must contain either only apples or only oranges.

### The Process
1. **Pick a fruit from the "Mixed" box:**
    * **If you pull an Apple:** You now know that this box contains only apples.
    * **If you pull an Orange:** You now know that this box contains only oranges.

2. **Determine the other boxes:**
    Let’s assume you pulled an **Apple** from the "Mixed" box:
    * Since you now know the "Mixed" box is actually the **Apples** box, there are two boxes left: the
one labeled "Oranges" and the one labeled "Apples."
    * The contents remaining are "Oranges" and "Mixed."
    * Look at the box labeled **"Oranges."** Since all labels are incorrect, this box cannot contain
oranges. Because it also cannot contain apples (you already found the apple box), it **must contain the
Mixed fruit.**
    * By process of elimination, the box labeled "Apples" **must contain the Oranges.**

***

**If you had pulled an Orange instead:**
* The "Mixed" box is the **Oranges** box.
* Look at the box labeled **"Apples."** It cannot contain apples (label is wrong) and it cannot contain
oranges (already found). Therefore, it **must contain the Mixed fruit.**
* By process of elimination, the box labeled "Oranges" **must contain the Apples.**

Output tokens: 1504 | Thinking tokens: 1117 | Seconds: 69.3
```

What should we notice?

This time, we are using a problem that requires several logical deductions rather than a simple calculation. We also set `thinking_level="high"` to give the model a higher thinking allowance.

Check the `thinking_tokens` value and compare it with the response.

If the value is greater than `0`, the API reported that the model used thinking tokens for this request.

If it is still `0`, that means **no thinking-token usage was reported by our current setup**. It does not necessarily mean that the model performed no internal processing.

The main point is that **thinking level, visible reasoning, and reported thinking tokens are separate things**. We can configure the model for additional thinking, but the API must also report that usage for us to measure it.

In [10]:
```
# Run the same direct prompt several times.
# We do this because response time and reported usage can vary between requests.

for run in range(3):

    reply, usage = ask_with_usage(
        direct_prompt,
        {"problem": hard}
    )

    print(
        f"Run {run + 1}: "
        f"reply={reply[:40]!r} | "
        f"output tokens={usage['output_tokens']} | "
        f"thinking tokens={usage['thinking_tokens']} | "
        f"seconds={usage['seconds']:.1f}"
    )
```

Out [10]:
```
Run 1: reply='1360' | output tokens=4 | thinking tokens=0 | seconds=43.7
Run 2: reply='1360' | output tokens=4 | thinking tokens=0 | seconds=0.9
Run 3: reply='1360' | output tokens=4 | thinking tokens=0 | seconds=0.9
```

What just happened: We sent the same direct prompt three times and recorded the usage reported each time.

Compare the three runs:

- Is the reply the same each time?
- Do the output tokens stay the same, or change?
- How much does the response time vary?
- Is `thinking_tokens` 0 in every run, or does it change between runs?

If `thinking_tokens` is 0 in all runs, no thinking-token usage was reported for these responses. This does **not** prove that the model performed no internal processing.

This small experiment shows why we should measure model behavior instead of assuming it. Even when the prompt stays the same, response time and reported usage can vary between requests.

Checkpoint: Your Week 4 classifier sorts messages into `billing`, `network`, `promo`, or `other`. Should you add "think step by step" to it? Why or why not?

Answer: Usually no. Sorting a message into one of four labels does not normally require several visible calculation steps, so adding a reasoning instruction may produce unnecessary output. However, the best way to evaluate that assumption is to test the classifier with and without the instruction and compare the results.

6. Controlling How the Reasoning Looks

Checkpoint: what are we adding, and why?

Sometimes you don't just want the model to show its reasoning, you want the response to follow a **consistent format**. In Week 4, you used few-shot examples to teach an output format. We can use the same idea with a **worked example**: put one solved problem inside a `ChatPromptTemplate`, then ask the model to solve a new problem using that layout.

The goal here is **format consistency**, not proof that the model's reasoning is correct. A worked example can show the model what the expected response should look like, but the resulting calculations still need to be checked.

Where you can use this: Customer-facing explanations ("here is how we calculated your bill"), tutoring apps that present solutions in a consistent format, and audit-friendly answers where a reviewer needs to check each line.

In [11]:

```
worked_example_prompt = ChatPromptTemplate.from_template(
    """Solve the problem below. Follow the same format as the example.

Problem: A pancit canton pack costs ₱18. Ana buys 5 packs and pays with a ₱100 bill. How much change do

Given: price per pack = 18, packs = 5, payment = 100

Step 1: Total cost = 18 x 5 = 90

Step 2: Change = 100 - 90 = 10

ANSWER: 10

Problem: {problem}"""
)


# Give the model a worked example, then ask it to solve a new problem.
reply = ask(
    worked_example_prompt,
    {"problem": problems[1]["problem"]}
)

print("MODEL REPLY:")
print(reply)

print()
print("EXPECTED:", problems[1]["answer"])
```

Out [11]:

```
MODEL REPLY:
Given: price per pack = 12, deal price = 30 for 3 packs, packs needed = 11

Step 1: Calculate the number of deals: 11 ÷ 3 = 3 deals with 2 packs remaining.

Step 2: Cost of the deals: 3 x 30 = 90

Step 3: Cost of remaining packs: 2 x 12 = 24

Step 4: Total cost: 90 + 24 = 114

ANSWER: 114

EXPECTED: 114
```

What just happened: The model should follow the `Given / Step / ANSWER` layout from the worked example. Compare this with the free-form reasoning in Section 2. The worked example helps control the **shape and level of detail** of the response.

Count the steps in the reply and compare them with the example, which used only two. The example guides the format, but it does not force the exact number of steps. This problem (Aling Nena's chips) needs a few more steps than the example does.

The fixed `ANSWER:` line is still useful because `extract_number()` can find the final result reliably. Check the reply against the `EXPECTED` value printed below it.

Lab Activity 5: Which Prompt Would You Ship?

Individual Activity

Scenario

You are building a **"How much will I pay?"** feature for a store app.

> ***Should the feature use the direct prompt or the step-by-step prompt?***

You will answer using **measurements and evidence**, not guesses.

Your tasks

1. **Build a mini test set** by writing one problem of your own.
2. **Run both prompts** on the three-problem set.
3. **Compare the results** using the measurements provided.
4. **Make a recommendation** based on your results.
5. **Research beyond the notebook** and find additional evaluation measures or considerations that could affect the decision.

> ***Important:*** *Your goal is not to prove that one prompt is always better. Your goal is to determine what your measurements tell you about this particular application.*

You may alter the code if you like, as long as you meet the guidelines above.

Step 1: Build Your Mini Test Set

Two problems are provided.

Replace the **third problem** with your own mathematical problem.

Your problem should require **more than one calculation step**, such as:

- a discount followed by a fee
- a fare calculation
- a tax followed by a payment
- a quantity calculation followed by a total cost
- a promotion followed by a remaining balance

Keep the answer key as a **Python expression**, like the other problems.

Two rules for your problem:

- The correct answer must be a **whole number**, or a peso amount with at most two decimal places. If needed, use `round(...)` in your answer key.
- Solve the problem **by hand** and confirm that it matches the number printed by the answer-key check below. An answer key that runs without errors can still be wrong, so the code cannot catch a key that does not match your problem.

In [ ]:

```
lab_problems = [
    {
        "problem": "A canteen sells rice meals at ₱45 and drinks at ₱15. A group buys 6 meals and 4 dri
                   "A 10% discount is applied to the total, then a ₱20 coupon is subtracted. "
                   "How much does the group pay, in pesos?",
        "answer": (6 * 45 + 4 * 15) * 0.9 - 20,
    },
    {
        "problem": "A delivery fee is ₱49 for the first 3 km and ₱10 for every additional km. "
                   "An order is delivered 8 km away and the food costs ₱320. "
                   "A ₱30 voucher is applied to the delivery fee only. "
                   "What is the total amount to pay, in pesos?",
        "answer": 320 + (49 + 10 * (8 - 3)) - 30,
    },
    # TODO: write your own multi-step problem and its answer key below.
    {
        "problem": "TODO: write your own problem here.",
        "answer": None,  # TODO: replace None with a Python expression
    },
]

# Check your answer keys before spending API calls.
for i, item in enumerate(lab_problems, start=1):

    if item["answer"] is None or item["problem"].startswith("TODO"):
        raise ValueError(
            f"Problem {i} is not finished. Write your own problem and answer key first."
        )

    print(f"Problem {i} answer key: {item['answer']:g}")
```

Step 2: Run Both Prompts

This reuses `evaluate_numeric()`, `direct_prompt`, and `reasoning_prompt` from Section 4.

The same three problems will be given to both prompts so that we can make a fair comparison.

The evaluation records:

- Accuracy
- Output tokens
- Thinking tokens
- Response time

In [ ]:

```
lab_direct_acc, lab_direct_rows = evaluate_numeric(
    direct_prompt,
    lab_problems
)

lab_reason_acc, lab_reason_rows = evaluate_numeric(
    reasoning_prompt,
    lab_problems
)

summarize(
    "DIRECT",
    lab_direct_acc,
    lab_direct_rows
)

print()

summarize(
    "STEP-BY-STEP",
    lab_reason_acc,
    lab_reason_rows
)
```

Step 3: Turn the Measures Into a Decision

Accuracy is one important measure, but a development team may need to consider other factors before choosing a prompt.

The next cell summarizes several measurements that can help us compare the two approaches.

It also calculates **output tokens per correct answer** and estimates the additional output generated if the application handled **100,000 questions per month**.

In [ ]:

```
def average(rows, key):
    return sum(r[key] for r in rows) / len(rows)


def tokens_per_correct(rows):
    """Total output tokens spent for each correct answer."""

    correct = sum(r["correct"] for r in rows)

    if correct == 0:
        return None

    return sum(r["output_tokens"] for r in rows) / correct


MONTHLY_QUESTIONS = 100_000

print(f"{'Measure':<32}{'Direct':>12}{'Step-by-step':>15}")
print("-" * 59)

print(
    f"{'Accuracy':<32}"
    f"{lab_direct_acc:>12.0%}"
    f"{lab_reason_acc:>15.0%}"
)

print(
    f"{'Avg output tokens':<32}"
    f"{average(lab_direct_rows, 'output_tokens'):>12.0f}"
    f"{average(lab_reason_rows, 'output_tokens'):>15.0f}"
)

print(
    f"{'Avg thinking tokens':<32}"
    f"{average(lab_direct_rows, 'thinking_tokens'):>12.0f}"
    f"{average(lab_reason_rows, 'thinking_tokens'):>15.0f}"
)

print(
    f"{'Avg seconds':<32}"
    f"{average(lab_direct_rows, 'seconds'):>12.1f}"
    f"{average(lab_reason_rows, 'seconds'):>15.1f}"
)

d_tpc = tokens_per_correct(lab_direct_rows)
r_tpc = tokens_per_correct(lab_reason_rows)

print(
    f"{'Output tokens per correct answer':<32}"
    f"{str(round(d_tpc)) if d_tpc else 'n/a':>12}"
    f"{str(round(r_tpc)) if r_tpc else 'n/a':>15}"
)

extra = (
    average(lab_reason_rows, "output_tokens")
    - average(lab_direct_rows, "output_tokens")
) * MONTHLY_QUESTIONS

print()

print(
    f"Extra output tokens per month if we choose step-by-step "
    f"({MONTHLY_QUESTIONS:,} questions): {extra:,.0f}"
)
```

Discuss Your Results

Answer the following based on the results from your experiment.

1. Accuracy

Which prompt produced more correct answers on your three problems?

Did either prompt produce an incorrect answer? Refer to the `Expected` and `Got` values.

2. Output

How did the two prompts differ in average output tokens?

If one prompt produced more output, what did that additional output provide?

3. Response Time

How did the two prompts compare in average response time?

Why might response time matter in a customer-facing application?

4. Thinking Tokens

Did `thinking_tokens` show a value above 0 for either prompt?

If the value is 0, what does that tell you?

What does it **not** tell you?

5. Application Requirements

Would your answer change if the application had to **show the customer how the bill was computed**?

Explain why.

6. Beyond the Notebook

Research and name **two other evaluation measures or considerations** (not accuracy, output tokens, thinking tokens, or response time) that could affect the choice between the two prompts.

For each one, explain what it is, how you would measure it, and how it could change your recommendation. Say where you found it.

Make Your Decision

Based on your experiment, write one sentence recommending an approach for the store application.

> ***"For the _____ feature, we would use the _____ prompt because _____ and _____."***

Your reasons must refer to **at least two measurements or observations** from the experiment. You may also use a measure or consideration from Question 6.